AES Encryption Performance Benchmarking on the Raspberry Pi 5

Zach Agle
Dept. of ECE, Cal Poly Pomona
zwagle@cpp.edu

Abigail Jeffie
Dept. of ECE, Cal Poly Pomona
ajeffie@cpp.edu

Juliana Neemeh
Dept. of ECE, Cal Poly Pomona
jneemeh@cpp.edu

Jacob Dominguez
Dept. of ECE, Cal Poly Pomona
jacobd@cpp.edu

Abstract - This paper analyzes how architectural implementation choices affect AES encryption performance on the Raspberry Pi 5. We benchmark AES across three cipher modes (CBC, CTR, and GCM) and three key lengths (128, 192, and 256-bit) using two tools: cryptsetup and OpenSSL with Linux perf. Measured values include throughput (MB/s), CPU cycles, instructions executed, cycles per instruction (CPI), and cache miss rate. Results show that AES-CTR at 128-bit keys achieves the highest throughput at over 2 GiB/s. CBC decryption is nearly twice as fast as CBC encryption. A key finding is that cache miss rates differ significantly between tools (2.6–2.9% for cryptsetup vs. less than 0.01% for OpenSSL), which we attribute to the difference in test duration.

Index Terms - AES, encryption benchmarking, ARM Cortex-A76, CBC, CTR, CPI, GCM, OpenSSL, perf, Raspberry Pi 5.

INTRODUCTION

The goal of this project is to analyze how architectural implementation choices affect encryption performance on the Raspberry Pi 5. We benchmark hardware-accelerated AES on the Raspberry Pi 5 using two independent measurement methods: perf paired with OpenSSL and cryptsetup.

AES is a symmetric-key encryption algorithm used to secure digital information. It encrypts data by converting readable plaintext into unreadable ciphertext and decrypts it back using the same secret key. It is used in applications like online banking, messaging apps, and protecting sensitive files.

The measured values in this study include throughput (MB/s), CPU cycles, instructions executed, instructions per cycle, and cache miss rate. The rest of this paper is organized as follows. Section II provides background on AES. Section III describes the benchmark tools used. Section IV presents our results. Section V compares the data across tools. Section VI concludes.

BACKGROUND

AES is a symmetric-key encryption algorithm used to secure digital information. It encrypts data by converting readable plaintext into unreadable ciphertext, and decrypts it back using the same secret key. AES transforms plaintext into ciphertext through repeated rounds of substitution, shifting, mixing, and key-based transformations. It uses key lengths of 128, 192, or 256 bits, where longer keys provide stronger security.

As an example of how AES works: the plaintext "HELLO" encrypts to "XJQZP" and can be decrypted back to "HELLO" using the same key. AES is widely used in applications like online banking, messaging apps, and protecting sensitive files.

AES MODES

CBC (Cipher Block Chaining): encrypts each block by combining it with the previous block output before encryption. This chaining dependency makes CBC inherently serial.

CTR (Counter Mode): encrypts each block separately by combining a counter value with a unique nonce. Because blocks have no dependency on each other, CTR can be fully parallelized.

GCM (Galois/Counter Mode) extends CTR by adding a built-in authentication step that verifies data integrity alongside encryption in a single operation. GCM is the standard mode in modern network protocols such as TLS and SSH.

BENCHMARK TOOLS

cryptsetup

The first tool is cryptsetup benchmark, which measures AES encryption performance using different modes and key sizes. It was used to test CBC and CTR modes at key sizes of 128, 192, and 256-bit. A Python script was written to run cryptsetup while also collecting CPU performance

data using Linux perf. The data collected includes cycles, instructions, cache references and misses, and branch references and misses. The script converts CPU metrics into per-second rates and monitors system temperature.

perf stat + OpenSSL

The second tool uses OpenSSL to measure encryption speed via the perf stat command. It tests CBC, CTR, and GCM modes at key sizes of 128, 192, and 256-bit. The script systematically tests each AES mode and key size using OpenSSL's built-in speed tests while simultaneously collecting low-level CPU performance metrics using Linux perf. It cleans and processes the output to display both encryption throughput and normalized hardware statistics like CPU cycles, instructions, and cache behavior per second. System temperature is also monitored throughout.

Measured Values

The metrics collected across both tools are: throughput (MB/s), CPU cycles, instructions executed, cycles per instruction (CPI), cache miss rate (%), and branch miss rate (%).

RESULTS

Throughput (cryptsetup)

Table I shows the encryption and decryption speeds measured by cryptsetup for CBC and CTR modes. Throughput goes down as key length increases. CTR mode is faster than CBC for encryption because CTR can process blocks in parallel, while CBC must wait for the previous block before starting the next one.

TABLE I. AES THROUGHPUT – CRYPTSETUP BENCHMARK

Cipher	Encrypt (MiB/s)	Decrypt (MiB/s)
AES-128-CBC	1,141	2,264
AES-192-CBC	1,054	2,042
AES-256-CBC	947	1,832
AES-128-CTR	2,170	2,179
AES-192-CTR	1,896	1,897
AES-256-CTR	1,717	1,712

CBC decryption is almost twice as fast as CBC encryption. CTR encryption and decryption speeds are nearly equal across all key sizes.

Microarchitectural Counters (cryptsetup)

Table II shows the perf counter results from the cryptsetup runs. The CPU cycles per second stayed around 2.36 to 2.40 billion across all tests. CTR mode had a lower CPI than CBC (0.46–0.48 vs. 0.65–0.66). Cache miss rates were around 2.6–2.9% for both modes, and branch miss rates stayed below 0.5%.

TABLE II. PERF COUNTERS – CRYPTSETUP (PER SECOND, 2S WINDOW)

Cipher	Cycles/s (B)	CPI	Cache Miss%	Branch Miss%
AES-128-CBC	2.36	0.66	2.59%	0.43%
AES-192-CBC	2.40	0.65	2.55%	0.35%
AES-256-CBC	2.40	0.65	2.58%	0.33%
AES-128-CTR	2.40	0.46	2.93%	0.37%
AES-192-CTR	2.40	0.47	2.91%	0.35%
AES-256-CTR	2.40	0.48	2.89%	0.34%

Throughput (OpenSSL)

Table III shows the encryption and decryption speeds measured by OpenSSL for CBC, CTR, and GCM modes. Throughput goes down as key length increases. CTR mode is faster than CBC for encryption because CTR can process blocks in parallel, while CBC must wait for the previous block before starting the next one. GCM and CTR are relatively equal in values.

TABLE III. AES THROUGHPUT – OPENSSL BENCHMARK

Cipher	Encrypt (MiB/s)	Decrypt (MiB/s)
AES-128-CBC	1,911	3,494
AES-192-CBC	1,595	2,976
AES-256-CBC	1,367	2,559
AES-128-CTR	3,226	3,224
AES-192-CTR	2,781	2,782
AES-256-CTR	2,414	2,414
AES-128-GCM	2,243	2,208
AES-192-GCM	1,995	2,000
AES-256-GCM	1,817	1,842

Microarchitectural Counters (OpenSSL)

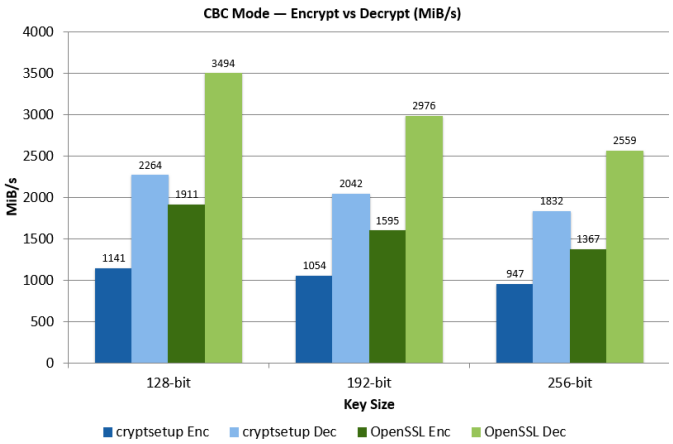
Table IV shows the perf results from the OpenSSL runs. The most surprising thing here compared to Table II is the cache miss rate. Where cryptsetup showed 2.6–2.9%, the OpenSSL runs showed essentially 0% across all modes and key sizes. We look at why this happens in Section V.

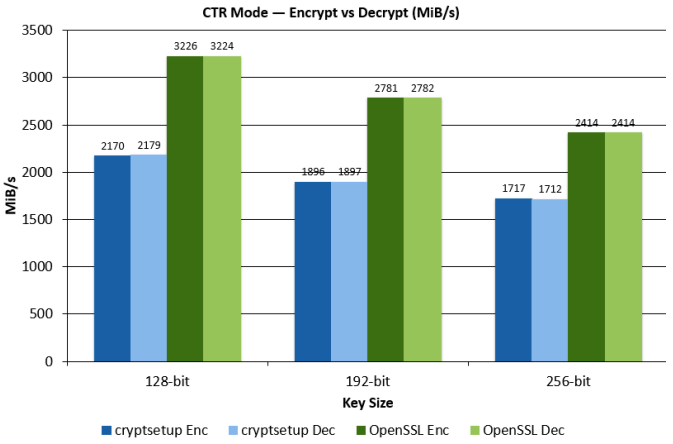
TABLE IV. PERF COUNTERS – OPENSSL (PER SECOND, 18S WINDOW)

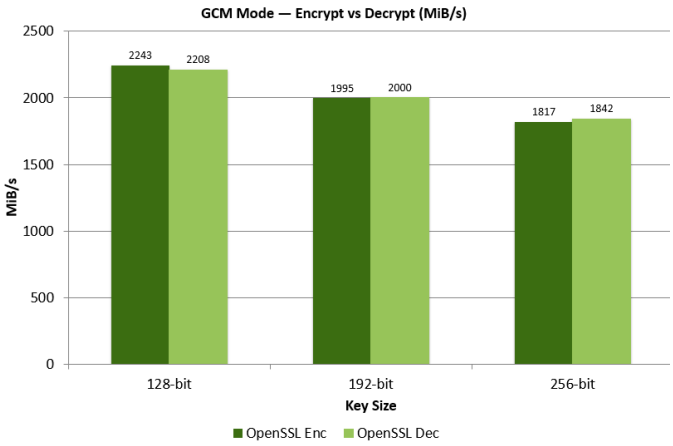
Cipher	Cycles/s (B)	CPI	Cache Miss%	Branch Miss%
AES-128-CBC	2.40	0.50	≈0.00%	0.01%
AES-192-CBC	2.40	0.48	≈0.00%	0.06%
AES-256-CBC	2.40	0.48	≈0.00%	0.01%
AES-128-CTR	2.40	0.33	≈0.00%	0.02%
AES-192-CTR	2.40	0.33	≈0.00%	0.02%
AES-256-CTR	2.40	0.34	≈0.00%	0.02%
AES-128-GCM	2.40	0.36	≈0.00%	0.28%
AES-192-GCM	2.40	0.35	≈0.00%	≈0.00%
AES-256-GCM	2.40	0.36	≈0.00%	≈0.00%

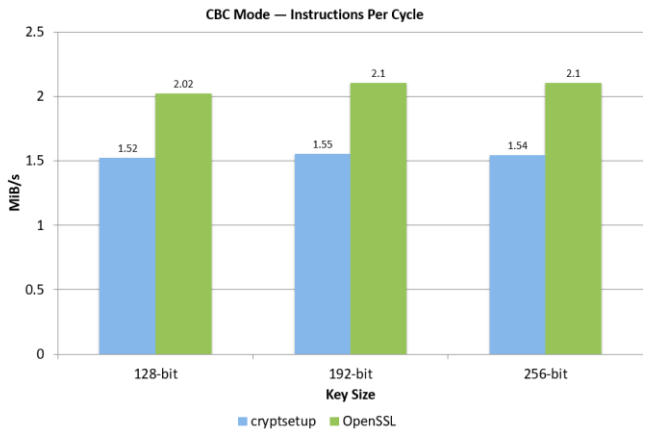
GCM mode CPI values (0.35–0.36) fall between CBC (0.48–0.50) and CTR (0.33–0.34). Cache miss rates for all OpenSSL modes were near zero (less than 0.01%), and branch miss rates were below 0.3% for most variants.

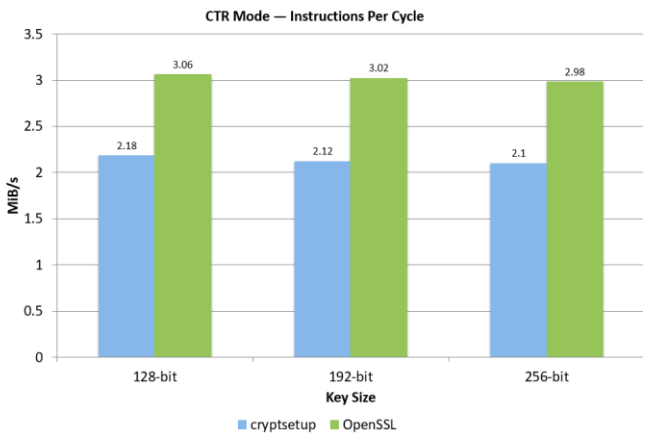
TABULATED RESULTS

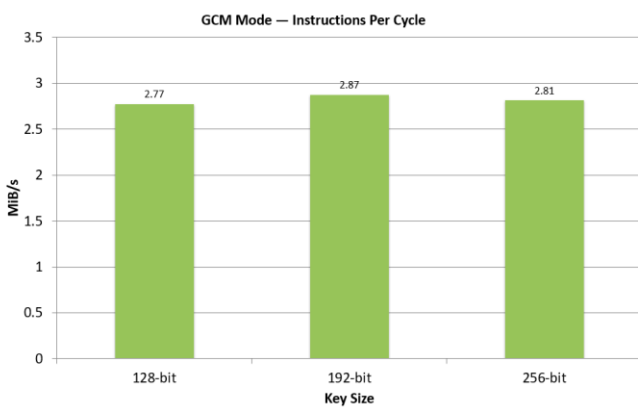












COMPARING DATA

Throughput

CTR is the fastest mode at over 2 GiB/s for 128-bit keys. CBC decrypt is roughly twice the speed of CBC encrypt across all key sizes. Throughput decreases as key size increases for both modes.

CPI

CPI values are consistent across both tools. CTR always has lower CPI than CBC, meaning it uses the processor more efficiently. GCM CPI falls between CBC and CTR. The cryptsetup CPI values are slightly higher than the OpenSSL values, which may be due to the shorter test window.

Cache Miss Rate

The most notable difference between the two tools is the cache miss rate. Cryptsetup showed 2.6–2.9% while OpenSSL showed less than 0.01% across all modes and key sizes. The cryptsetup tests run for about 2 seconds per variant while the OpenSSL tests run for 18 seconds, so the longer window allows the cache to fully warm up.

CONCLUSION

This project benchmarked AES encryption on the Raspberry Pi 5 across three cipher modes (CBC, CTR, GCM) and three key lengths (128, 192, 256-bit) using two tools: cryptsetup and OpenSSL with Linux perf. CTR mode at 128-bit keys was the fastest, reaching over 2 GiB/s. CBC decryption was nearly twice as fast as CBC encryption. GCM CPI values were between those of CBC and CTR. The biggest difference between tools was the cache miss rate, which was much higher for the shorter cryptsetup tests than for the longer OpenSSL tests.